Linux Shell and Basic Commands-

- **pwd**: Print working directory paths.
- **ls**: List contents of the current directory.
- **cd**: Change directories (cd .. moves back).
- **mkdir**: Create a brand new directory folder.
- **rmdir**: Remove an entirely empty directory.
- **cp**: Copy files or folders (cp source.txt backup.txt).
- **mv**: Move or rename a file or directory.
- **rm**: Delete a file from the disk space.
- **cat**: Display whole text file data to stdout.
- **echo**: Print strings or variable values to screen.
- **find**: Scan for files (find . -name "*.c").
- **history**: Output the terminal's execution log history.

```
  136  nano exec4.c
  137  gcc exec4.c -o exec4
  138  ./exec4
  139  nano exec5.c
  140  gcc exec5.c -o exec5
  141  nano exec5.c
  142  gcc exec5.c -o exec5
  143  sudo apt update
  144  sudo apt install git
  145  git --version
  146  history
  147  pwd
  148  ls
  149  cd
  150  mkdir
  151  rmdir
  152  cp
  153  mmv
  154  mv
  155  find
  156  echo
  157  shell usage
  158  ./exec1
  159  ./exec2
  160  ./exec3
  161  ./exec4
  162  ./exec5
  163  nano exec5.c
  164  gcc exec5.c -o exec5
  165  nano exec5.c
  166  gcc exec5.c -o exec5
  167  ./exec5
  168  nano exec6.c
```

5. Process Creation using fork() - Process abstraction, fork(), parent and child processes, getpid(), getppid(), multiple fork() examples

```
==> fork_demo.c <==
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork(); // Fork returns twice on success

    if (pid < 0) {
        perror("Fork execution failed");
        return 1;
    } else if (pid == 0) {
        // Child Process execution loop
        printf("[CHILD] PID: %d, Parent PPID: %d\n", getpid(), getppid());
    } else {
        // Parent Process execution loop
        printf("[PARENT] PID: %d, Generated Child PID: %d\n", getpid(), pid);
    }
    return 0;
}
```

6.Process Replacement using the exec() Family- execl(), execv(), execvp(), replacing the current process, executing Linux commands from C

```
==> exec_family.c <==
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Replaced process via execlp()...\n");
    // System utility execution swap routine
    execlp("ls", "ls", "-l", NULL);

    perror("This point is only reachable if execution errors occur");
    return 1;
}
```

```
==> mini_shell.c <==
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[100];

    while(1) {
        printf("mini-shell> ");
        if (!fgets(command, sizeof(command), stdin)) break;

        // Strip trailing newline character
        command[strcspn(command, "\n")] = 0;
        if (strcmp(command, "exit") == 0) break;

        if (fork() == 0) {
            // Child Worker executes parsed instruction token
            char *args[] = {command, NULL};
            execvp(command, args);
            perror("Execution Failure");
            exit(1);
        } else {
            wait(NULL); // Hold shell window prompt open
        }
    }
    return 0;
}
```

```
==> wait_sync.c <==
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    if (fork() == 0) {
        printf("[CHILD] Running brief background calculations...\n");
        sleep(2);
        printf("[CHILD] Task complete. Terminating process context.\n");
    } else {
        printf("[PARENT] Halting loop until child process drops out...\n");
        wait(NULL); // Synchronizes execution sequence timeline
        printf("[PARENT] Child finished cleanly. Continuing host pipeline.\n"
);
    }
    return 0;
}
```

```
mohana@MOHANA:~$     nano wait_sync.c
mohana@MOHANA:~$     gcc wait_sync.c -o wait_sync

mohana@MOHANA:~$
mohana@MOHANA:~$     ./wait_sync
[PARENT] Halting loop until child process drops out...
[CHILD] Running brief background calculations...
[CHILD] Task complete. Terminating process context.
[PARENT] Child finished cleanly. Continuing host pipeline.
mohana@MOHANA:~$     nano mini_shell.c
mohana@MOHANA:~$     gcc mini_shell.c -o mini_shell
mohana@MOHANA:~$     ./mini_shell
mini-shell> ls
all_codes_and_outputs.txt   exec5.c         gcc_program10.c   gcc_program6.c
exec1                       exec6.c         gcc_program2      gcc_program7
exec1.c                     exec_family     gcc_program2.c    gcc_program7.c
exec2                       exec_family.c   gcc_program3      mini_shell
exec2.c                     fork_demo       gcc_program3.c    mini_shell.c
exec3                       fork_demo.c     gcc_program4      pro
exec3.c                     gcc_progl       gcc_program4.c    pro.c
exec4                       gcc_program1    gcc_program5      wait_sync
exec4.c                     gcc_program1.c  gcc_program5.c    wait_sync.c
exec5                       gcc_program10   gcc_program6
mini-shell> pwd
/home/mohana
mini-shell> date
Sat Aug  1 03:59:09 UTC 2026
mini-shell>
```

```
mohana@MOHANA:~$     ./fork_demo
[PARENT] PID: 1312, Generated Child PID: 1313
[CHILD] PID: 1313, Parent PPID: 1312
mohana@MOHANA:~$     nano exec_family.c
mohana@MOHANA:~$     gcc exec_family.c -o exec_family
mohana@MOHANA:~$     ./exec_family
Replaced process via execlp()...
total 344
-rw-r--r-- 1 mohana   mohana    1077 Aug  1 03:12 all_codes_and_outputs.txt
-rwxr-xr-x 1 mohana   mohana   15992 Aug  1 03:53 exec1
-rw-r--r-- 1 mohana   mohana     191 Jul 31 09:00 exec1.c
-rwxr-xr-x 1 mohana   mohana   15992 Aug  1 03:53 exec2
-rw-r--r-- 1 mohana   mohana     187 Jul 31 09:21 exec2.c
-rwxr-xr-x 1 mohana   mohana   16048 Aug  1 03:53 exec3
-rw-r--r-- 1 mohana   mohana     214 Jul 31 09:21 exec3.c
-rwxr-xr-x 1 mohana   mohana   16048 Aug  1 03:53 exec4
-rw-r--r-- 1 mohana   mohana     211 Jul 31 09:22 exec4.c
-rwxr-xr-x 1 mohana   mohana   16120 Aug  1 03:53 exec5
-rw-r--r-- 1 mohana   mohana     274 Aug  1 03:09 exec5.c
-rw-r--r-- 1 mohana   mohana       0 Aug  1 03:10 exec6.c
-rwxr-xr-x 1 mohana   mohana   16048 Aug  1 03:55 exec_family
-rw-r--r-- 1 mohana   mohana     275 Aug  1 03:55 exec_family.c
-rwxr-xr-x 1 mohana   mohana   16128 Aug  1 03:54 fork_demo
-rw-r--r-- 1 mohana   mohana     508 Aug  1 03:53 fork_demo.c
-rwxr-xr-x 1 mohana   mohana   16000 Jul 31 09:00 gcc_progl
-rwxr-xr-x 1 mohana   mohana   16008 Jul 30 09:33 gcc_program1
-rw-r--r-- 1 mohana   mohana     103 Jul 30 09:33 gcc_program1.c
-rwxr-xr-x 1 mohana   mohana   16520 Jul 30 16:17 gcc_program10
-rw-r--r-- 1 mohana   mohana    1138 Jul 30 16:17 gcc_program10.c
-rwxr-xr-x 1 mohana   mohana   16000 Jul 30 09:34 gcc_program2
-rw-r--r-- 1 mohana   mohana      99 Jul 30 09:34 gcc_program2.c
-rwxr-xr-x 1 mohana   mohana   16000 Jul 30 09:34 gcc_program3
-rw-r--r-- 1 mohana   mohana     167 Jul 30 09:34 gcc_program3.c
-rwxr-xr-x 1 mohana   mohana   16088 Jul 30 09:35 gcc_program4
-rw-r--r-- 1 mohana   mohana     283 Jul 30 09:35 gcc_program4.c
-rwxr-xr-x 1 mohana   mohana   16040 Jul 30 09:35 gcc_program5
-rw-r--r-- 1 mohana   mohana     219 Jul 30 09:35 gcc_program5.c
-rwxr-xr-x 1 mohana   mohana   16048 Jul 30 09:36 gcc_program6
-rw-r--r-- 1 mohana   mohana     120 Jul 30 09:36 gcc_program6.c
-rwxr-xr-x 1 mohana   mohana   16040 Jul 30 09:36 gcc_program7
-rw-r--r-- 1 mohana   mohana     224 Jul 30 09:36 gcc_program7.c
```